What Is a Benchmark? A Beginner Explanation

ClusterAI-HPC – CDAC HPC-AI Project

A **benchmark** is a fair test that measures how fast or how well something performs, so you can compare it with something else using real numbers instead of guesses.

Everyday example

Suppose you want to know which of two phones is faster. You give both the same task, such as opening the same 10 apps, and time each one. Phone A takes 20 seconds and Phone B takes 35 seconds. That timed test is a benchmark. It only means something because the task was identical for both.

Example from your project

Say you write a program that adds up 1 billion numbers, and you make three versions of it:

- **Serial:** one CPU core does all the work
- **OpenMP:** 4 CPU cores share the work inside one computer
- **MPI:** 4 separate processes share the work (they can even run on different computers)

You run all three on the **same task** and record how long each takes. The numbers below are made up to show the idea. Your real numbers will come from your own runs.

Version	Time	Speedup (T1 / T)	Efficiency (speedup / cores)
Serial (1 core)	40 s	1.0	100%
OpenMP (4 cores)	12 s	3.3	83%
MPI (4 processes)	13 s	3.1	77%

- **Speedup** tells you how many times faster the parallel version is than the serial one. 40 / 12 is about 3.3 times faster.
- **Efficiency** tells you how well the extra cores were used. Perfect use of 4 cores would give 4 times faster, or 100%. Getting 3.3 out of 4 is 83%.

Why your project has a benchmarking module

The point of HPC is to make heavy computing faster by using many cores or machines. A benchmark is how you prove that it actually works, and how much it helps. Your dashboard will run these tests through Slurm, save the times, and draw the speedup and efficiency graphs.

Rules for a fair benchmark

1. Use the **same task and the same input size** for every version.
2. Run each test **several times** and take the average, because a single run can be unusually slow or fast.
3. Change **one thing at a time**, such as only the number of cores.
4. Report what you **actually measured**, even if the result is disappointing. Small test machines often show weak scaling, and you can explain why in your report.